MSc(IT) Semester III [Autumn 2025-26]
IT644: Web Services & SOA

Date: Nov 22, 2025 Duration: 2 hours Max. Marks: 60

- All questions are compulsory and each MCQ is of 1 mark.
- Select all choices in MCQ that apply, wherever needed.

ID: ________________ Name: ________________________________

1. Which of the following are key steps when migrating from monolithic to a microservices model?

   A Ensure centralized logging and monitoring.

   B Implement APIs for communication between services.

   C Use a database-per-service architecture.

   D Adopt a big-bang approach to migration.

   E Identify and modularize business capabilities.

2. The Adapter Pattern is necessary when you want to reuse an existing class (Adaptee) in a new inheritance hierarchy, but the methods of the existing class do not have the same signature as those in the new hierarchy. What technique does the Adapter class use to integrate the Adaptee?

   A The Adapter class directly incorporates the Adaptee via inheritance.

   B The Adapter class creates a simplified public view of the Adaptee package.

   C The polymorphic methods of the Adapter delegate to methods of the Adaptee, which is connected by an association.

   D The Adapter class implements the Observer interface so it can be notified when the Adaptee changes.

   E The Adapter class uses a private constructor and a static getInstance() method.

3. A developer wants to model a student who can change their current status throughout their academic life, specifically their Attendance Role (Full-Time or Part-Time) and their Level Role (Graduate or Undergraduate). Based on the Player-Role Pattern, and assuming a student must possess one status in each category at all times, what are the appropriate multiplicity settings for the associations linking the << *Player* >> (Student) to the two abstract << *Role* >> superclasses?

   A One-to-one (1:1) multiplicity for both associations utilizing two separate role superclasses.

   B Zero-to-one (0..1:1) multiplicity for the Attendance Role and one-to-one (1:1) multiplicity for the Level Role.

$\boxed{\text{C}}$ The General Hierarchy pattern should be used instead of Player-Role.

$\boxed{\text{D}}$ One-to-many (1:) multiplicity for both associations, allowing multiple roles in each category.

$\boxed{\text{E}}$ One-to-many (1:) multiplicity to one abstract role superclass, and the other status must be modeled as an ordinary subclass of Student.

4. In the context of using the Player-Role Design Pattern in development of mobile apps, which of the following are the correct ways to implement this pattern effectively?

$\boxed{\text{A}}$ Separate concerns by implementing a RoleManager that dynamically assigns and manages roles based on runtime conditions.

$\boxed{\text{B}}$ Implement roles as independent classes that are composed into player objects dynamically at runtime.

$\boxed{\text{C}}$ Use interfaces or abstract classes to define role behaviors and allow players to adopt multiple roles through polymorphism.

$\boxed{\text{D}}$ Hardcode player-role relationships using conditionals in the player class.

$\boxed{\text{E}}$ Design roles with direct references to the player class for tight coupling to streamline role-specific operations.

5. A high-growth e-commerce startup currently uses a monolithic architecture and is debating switching to microservices. Which specific combination of conditions/circumstances would make the transition to microservices the most advantageous choice for the company?

$\boxed{\text{A}}$ The company is satisfied with its current product development time, and only needs to scale its database.

$\boxed{\text{B}}$ The company needs to quickly launch an app with basic functionality and can only afford one small development team.

$\boxed{\text{C}}$ The company wants to strictly adhere to a single defined technology stack for all current and future features.

$\boxed{\text{D}}$ The company expects high-pace growth, wants to modernize its application fast, and strives to develop a highly reliable application.

$\boxed{\text{E}}$ The company's primary issue is an unoptimized development process and insufficient business analysis.

6. If the General Hierarchy pattern is implemented such that the subordinates association linking a SuperiorNode back to the abstract Node superclass has a many-to-many multiplicity, what critical impact does this have on the resulting structure of instances?

$\boxed{\text{A}}$ It restricts the hierarchy to having only one top node that has no superiors.

$\boxed{\text{B}}$ It indicates that the Node class must implement the ReadOnlyInterface.

C It forces the use of the Singleton pattern for the top-level node.

D It means that the hierarchy of instances becomes a lattice, allowing a node to have multiple superiors.

E It necessitates that all nodes must also be instances of NonSuperiorNode.

7. A company attempting to migrate its legacy monolithic application to microservices is concerned about the "Distributed Monolith" outcome. Why is this outcome undesirable even though the application has been split into separate services?

A The company failed to use an appropriate CIAM platform like AWS Cognito or Auth0.

B It implies that the newly extracted services are highly connected or heavily dependent on the monolith, meaning the original complexity and maintenance problems remain unsolved.

C It means the application has failed in first step of migration by not addressing poor team management.

D The use of different technologies for each new microservice leads to excessive system complexity.

E The new microservices are too small to be efficiently tested by the QA team individually.

8. In the Hub-and-Spoke integration architectural model, what element is responsible for handling the connections between all subsystems so that the subsystems do not communicate directly?

A The Enterprise Service Bus (ESB) messaging backbone

B Application Programming Interfaces (APIs) realizing direct Point-to-Point connections

C A central hub (message broker) and integration engine

D A dedicated adapter and integration engine for each system

E Electronic Data Interchange (EDI) via a value-added network (VAN)

9. In the context of implementing a singleton design pattern for contemporary web-service APIs, which of the following are correct practices to ensure effectiveness and maintainability?

A Create multiple instances of the singleton class to support horizontal scaling.

B Implement lazy initialization to create the instance only when it is first accessed.

C Store configuration or shared resources in the singleton to avoid redundancy across API calls.

D Use a private constructor and provide a public static method to access the singleton instance.

E Use synchronization or thread-safe mechanisms to handle multiple threads attempting to access the singleton instance.

10. A software designer is creating a class diagram for a library system. They want to avoid duplicating

common information for all physical copies of the same book (e.g., Title and Author) for all same books in the library. Which pattern are they using, and what is the required association multiplicity between such classes?

- $\boxed{\text{A}}$ Abstraction-Occurrence Pattern; many-to-one association (:1).
- $\boxed{\text{B}}$ Observer Pattern; many-to-many association (:).
- $\boxed{\text{C}}$ General Hierarchy Pattern; optional-to-many association (0..1:).
- $\boxed{\text{D}}$ Abstraction-Occurrence Pattern; one-to-many association (1:).
- $\boxed{\text{E}}$ Player-Role Pattern; one-to-one association (1:1).

11. Which major characteristic of microservices applications means that, even if one service fails, the entire application may lack certain functionality but will keep working without critical issues?

- $\boxed{\text{A}}$ Failures resistance
- $\boxed{\text{B}}$ Simplified onboarding
- $\boxed{\text{C}}$ Decentralized approach
- $\boxed{\text{D}}$ Great evolutionary capacity
- $\boxed{\text{E}}$ Continuous delivery

12. Which of the following factors are critical to evaluating the scalability of web applications?

- $\boxed{\text{A}}$ Load balancing mechanisms.
- $\boxed{\text{B}}$ Vertical and horizontal scaling capabilities.
- $\boxed{\text{C}}$ User interface design principles.
- $\boxed{\text{D}}$ Database optimization techniques.
- $\boxed{\text{E}}$ Test coverage for unit testing.

13. A company needs to ensure its newly developed customer dashboard can seamlessly pull customer contact details from its existing, non-replaceable Customer Relationship Management (CRM) system, which is based on outdated technology. This integration is crucial for day-to-day workflow. What type of system integration is being performed?

- $\boxed{\text{A}}$ Legacy system integration, connecting modern applications to existing, outdated, and critical systems.
- $\boxed{\text{B}}$ Data Integration, gathering data into one storage location for a unified view.
- $\boxed{\text{C}}$ Business-to-business (B2B) integration, automating document exchange across two organizations.
- $\boxed{\text{D}}$ Enterprise Application Integration (EAI), focusing on unifying different subsystems inside one

business environment.

E Third-party system integration, expanding functionality with external tools like payment gateways.

14. A company is migrating from a monolith to microservices. After thoroughly analyzing project issues and optimizing the monolith, what must the team do immediately before selecting a complete versus incremental rewrite approach?

A Extract the most important features to create a strangler application.

B Start implementing new features as distinctive microservices.

C Identify whether team management or insufficient business analysis caused the initial project problems.

D Analyze possible solutions to see if project issues can be solved without converting the monolith.

E Develop a specialized authentication service.

15. A critical transaction process in a financial app is tested & found to have an average response time of 2.5 seconds. Based on response time metrics, what is the most likely immediate user reaction?

A The user will certainly notice the delay, but their concentration and mental flow regarding the task are unlikely to be fully broken or lost.

B The user will perceive the transaction as instantaneous and highly efficient.

C The user's flow of thought will be completely interrupted, likely leading to task abandonment.

D The system is meeting recommended threshold for most user-centric performance reqs.

E 55% of users will abandon the task because 2.5 seconds exceeds the optimal 1-second load time.

16. In the context of quality attributes, Maintainability is defined as:

A How well the system and its data are protected against attacks

B How much time it takes to fix the issue when it arises

C How often the system experiences critical failures

D How long the average system downtime is

E How fast the system returns results when under maintenance

17. Quality attributes (QAs) are often concretized and decomposed as scenarios. What is the underlying reason why this solution (using scenarios) is necessary for QAs?

A QAs are concerned exclusively with run-time behavior, making them highly sensitive to specific operational triggers (stimuli).

B QAs are typically defined by the user and are mandatory, requiring structured verification.

C QAs tend to be hard to define and articulate, are often described vaguely, and may be neglected initially or hidden within other requirements

D QAs can only be quantified using metrics like Mean Time to Restore System (MTTRS), which inherently require scenarios.

E QAs are solely focused on architectural design aspects, making them hard to test without scenarios.

18. Which related attribute of portability ensures that the software can be used instead of another designated product for the same purpose, producing the same outcomes on all target platforms?

A Co-existence or Compatibility

B Adaptability

C Interoperability

D Replaceability

E Installability

19. Suppose you have a highly complex, multi-component computation object (Forecaster) that generates output used by many display and calculation modules (WeatherViewer, Scheduler). Since the Forecaster should not need to know the specific classes of the modules it updates, the Observer Pattern is applied, making the Forecaster the $<< ConcreteObservable >>$. What critical design benefit, related to system flexibility, does this configuration provide?

A It maximizes system flexibility by allowing the Forecaster to communicate with other objects without needing to know which concrete class they belong to.

B It ensures that the Forecaster object is a globally accessible Singleton instance.

C It restricts the computational output to only one specific receiver, improving security.

D It allows the Forecaster to act as an Adapter, wrapping an existing unrelated computational class.

E It guarantees that the Forecaster is an immutable object whose state cannot be changed after creation.

20. Which of the following statements accurately describes the primary objective of Non-functional Requirements (NFRs)?

A Describe what the product does

B Define product features

C Describe how the product works

D They are mandatory

E Focus on user requirements

21. A major financial services application, built on a microservices architecture, needs to ensure that data traveling between its 'Credit Check' service and 'Transaction Processing' service is absolutely secure. Which of the following combination of security best practices, offers the most robust solution for securing this communication channel?

A Developing a custom-built solution for every service's authentication and authorization.

B Using shared session storage for ID verification and following Domain-driven design (DDD) guidelines.

C Establishing in-transit data encryption using HTTPS and utilizing a service mesh for validation.

D Avoiding hardcoding login credentials and configuring an API Gateway.

E Utilizing the principle of least privilege and implementing JSON Web Tokens (JWT).

22. How does the microservices approach differ from monolithic architecture in addressing application scaling requirements?

A Microservices require the entire system to be scaled simultaneously to meet the highest performing service's needs.

B Each microservice can be scaled individually, following its unique performance requirements.

C Scaling is only managed via the addition of new microservices, not resource expansion.

D Microservices require scaling the application database only.

E Monolithic architecture provides outstanding flexibility for scaling individual components.

23. A technology company wants to modernize its large, monolithic application. Company requires the ability to use different programming languages & technology stacks for various parts of the system to achieve specific business goals. Which benefit of microservices directly addresses this requirement?

A Improved continuous delivery

B High sustainability

C Fail-proof design

D Simplified onboarding

E Technical variety

24. The Facade Pattern aims to simplify the use of a complex package by creating a special class, the $<< Façade >>$. What secondary advantage does this pattern provide regarding the modification of the complex package?

$\boxed{\text{A}}$ It ensures the package remains a Singleton, accessible only through one entry point.

$\boxed{\text{B}}$ It guarantees that the internal package classes are now immutable.

$\boxed{\text{C}}$ It allows the $<< Façade >>$ to inherit from multiple abstract classes to incorporate the internal functionality.

$\boxed{\text{D}}$ Any modifications made to the internal package should only necessitate redesign of $<< Façade >>$ class, not classes in other packages.

$\boxed{\text{E}}$ It automatically turns the package into an Observable that notifies external classes of changes.

25. A development team is establishing the Availability requirement for a mission-critical financial service. The goal is to maximize the time the system is accessible, but they understand that no application is failure-proof. They have defined the system's target reliability and maintainability metrics. Which definition of Availability best aligns with quantifying this business-critical requirement?

$\boxed{\text{A}}$ Determining the percentage of time the system is accessible for operation during a specific time period (e.g., 99.98% availability during business hours).

$\boxed{\text{B}}$ Calculating the Mean Time to Restore the System (MTTRS) after a failure event.

$\boxed{\text{C}}$ Quantifying the hardware, operating systems, and browsers supported by the application.

$\boxed{\text{D}}$ Counting the number of critical bugs found in production over the past month.

$\boxed{\text{E}}$ Defining the maximum time (e.g., 10 seconds) the user must wait before a transaction is processed.

26. A development team is choosing an authentication method for a new microservices application and is considering using JSON Web Tokens (JWT). Which of the following is identified as a disadvantage of using JWT?

$\boxed{\text{A}}$ It requires the usage of one central database for user verification.

$\boxed{\text{B}}$ Basic user information stored in the token is not always up-to-date.

$\boxed{\text{C}}$ High load is placed on the authentication service, potentially creating a bottleneck.

$\boxed{\text{D}}$ It promotes strict dependencies between authentication and resource services.

$\boxed{\text{E}}$ The authentication service may become a performance bottleneck due to shared storage use.

27. To protect data integrity, a system uses Read-Only Interface Pattern. The MutablePerson class contains sensitive data and implements Person interface, which is the $<< ReadOnlyInterface >>$. An

$<< UnprivilegedClass >>$ receives an instance of Person interface. If the $<< UnprivilegedClass >>$ attempts to call a method that modifies the internal attributes of the object (a method present only in MutablePerson), what prevents this operation from succeeding?

$\boxed{\text{A}}$ The $<< UnprivilegedClass >>$ is only passed an instance of the Person interface, which by definition only contains read-only operations.

$\boxed{\text{B}}$ The private constructor of MutablePerson prevents modification outside the package.

$\boxed{\text{C}}$ The $<< UnprivilegedClass >>$ must delegate its access to a specialized Adapter class.

$\boxed{\text{D}}$ The MutablePerson instance automatically notifies the $<< UnprivilegedClass >>$ if an unauthorized modification is detected.

$\boxed{\text{E}}$ The object is fundamentally immutable, meaning no method call can change its internal state.

28. What specific issue or concern, known as a 'Force,' needs to be considered when solving a problem using a design pattern, according to the standard template?

$\boxed{\text{A}}$ The context in which the pattern applies.

$\boxed{\text{B}}$ Acknowledgements of those who developed or inspired the pattern.

$\boxed{\text{C}}$ Solutions that are inferior or do not work in this context (Antipatterns).

$\boxed{\text{D}}$ The recommended way to solve the problem.

$\boxed{\text{E}}$ The issues or concerns that you need to consider when solving the problem, including criteria for evaluating a good solution.

29. An enterprise is currently using a complex internal architecture consisting of eight deeply interconnected, proprietary software modules. The IT team frequently updates and replaces these modules. They have found that using a Point-to-Point (P2P) model resulted in a complex "star/spaghetti" mess, making maintenance and upgrades difficult. To maximize high scalability, minimize system decoupling, and allow modules to be replaced or changed without affecting others, which integration architecture should the enterprise adopt?

$\boxed{\text{A}}$ Hub-and-Spoke model, leveraging its security benefits and centralized integration engine.

$\boxed{\text{B}}$ Data Integration model, focusing solely on gathering data into a single storage location.

$\boxed{\text{C}}$ Point-to-point (P2P) model, as it is quick to build for small-scale integrated systems.

$\boxed{\text{D}}$ Enterprise Service Bus (ESB) model, because each subsystem is decoupled by a "messaging bus," favoring high scalability and easy replacement.

$\boxed{\text{E}}$ Hybrid Integration Platform (HIP), to connect on-premise and cloud solutions.

30. Which of the following accurately contrasts the two main types of system scaling?

$\boxed{\text{A}}$ Vertical scaling is more reliable than horizontal scaling because it runs the application on a more powerful infrastructure.

$\boxed{\text{B}}$ Vertical scaling is typically preferred for highly scalable microservices architectures.

$\boxed{\text{C}}$ Vertical scaling involves higher upfront hardware costs & requires re-architecting a software system.

$\boxed{\text{D}}$ Horizontal scaling necessitates bringing entire system down for upgrades, unlike vertical scaling.

$\boxed{\text{E}}$ Horizontal scaling is generally more reliable than vertical scaling because it involves multiple machines sharing the computational workload, enhancing fault tolerance.

End of MCQs Section

Theory Section begins next page

1. A university is launching a hybrid mobile + web platform for examination management. The system supports online exams, proctoring, student identity verification, and real-time analytics. Students use a mobile app or browser; teachers use a web dashboard. The system integrates:

   - A cloud-hosted microservice cluster (Exam, Authentication, Proctoring, and Analytics Service).
   - A third-party AI-based video-proctoring API
   - A third-party SMS/Email notification service.
   - A secure on-premise database server at the university.

   Design a UML deployment diagram that shows the nodes, devices, containers, and communication channels. Include elements such as app containers, load balancers, cloud VMs or Kubernetes nodes, external API endpoints, and security considerations. The diagram should clearly model how mobile devices, browsers, cloud microservices, and on-premise servers interact during an exam session. (4 Marks)

2. A health-tech startup is building a tele-consultation app (Android, iOS, and Web). The backend runs on serverless cloud functions and persistent data is stored in a managed NoSQL database. The system integrates following components:

   - Appointment Scheduling, and Video Consultation Component
   - E-Prescription Generator
   - Payment & Billing Component and Third-party Payment Gateway
   - Third-party Real-Time Video API
   - Notification Component (push + email) and User Profile Manager

   Prepare a UML Component Diagram that captures the following: (a) Application-level components (b) Connectors and interfaces between components (c) Provided/required interfaces for external third-party APIs (d) Distinction between presentation-layer components, domain-layer services, and data-access mechanisms (e) Serverless architecture constraints (statelessness, event triggers). (4 Marks)

3. Design the interaction flow for a food-delivery cross-platform app (web + mobile) when a user places an order and tracks it in real time. The architecture uses cloud-hosted microservices and third-party services. The scenario includes the following actors and components:

   User (via mobile/web app), API Gateway, Authentication Service, Restaurant Service, Order Service, Payment Service, Delivery Assignment Service, Real-time Tracking Service (WebSocket-based), Third-party Maps & Geolocation API, Third-party Payment Provider, and Delivery Partner App

   Model the UML Sequence Diagram showing the chronological interactions for:

   1. User login, Menu fetching, Order creation, & Payment processing via a third-party gateway.
   2. Automatic delivery assignment through microservice calls
   3. Live location updates using WebSockets and a third-party maps API
   4. Notification triggers at key points (order accepted, food prepared, out for delivery)

   The diagram must accurately reflect asynchronous interactions, callbacks/webhooks, and microservice-to-microservice communication patterns. (4 Marks)

4. We are interested in capturing various scenarios for a web app in the process of its overall architecture design. Starting with the context of web app being developed along with its stakeholder concerns till a validated architecture is realized, depict the entire process of architectural evolution/development using a suitable diagram. (4 marks)

5. Visualize general-hierarchy design pattern using a UML diagram instantiating with an example. (4 marks)

6. Complete the following table. (3 marks)

Feature	**SOAP**	**REST**
Data Format		
Statefulness		
Complexity		
Performance		
Security		

7. Complete the following table. (3 marks)

Feature	**Web Server**	**Application Server**
Primary Role		
Layer		
Content		
Protocols Supported		
Task Focus		
Scaling		

8. Summarize industry best practices to document quality attributes (QAs). (2 marks)

Ans:

9. Define the Security quality attribute with examples. (2 marks)

Ans: